

20230533 김양현 컴퓨터운영체제 12주차 과제

[복합문제 1] 파일 경로명을 통한 i-node 탐색 과정

- 질문 예시: Unix 파일 시스템에서 경로명 /usr/dev/source/app.c 파일에 접근할 때, 거쳐야 하는 i-node들의 순서는 어떻게 되는가?
- 정답: 파일을 읽기 위해서는 총 5개의 i-node를 순서대로 읽어야 합니다.
 1. /(루트 디렉터리)의 i-node
 2. /usr 디렉터리의 i-node
 3. /usr/dev 디렉터리의 i-node
 4. /usr/dev/source 디렉터리의 i-node
 5. /usr/dev/source/app.c 파일의 i-node

핵심 메커니즘 (해설): Unix 파일 시스템에서 수퍼 블록은 루트 디렉터리(/)의 i-node 번호를 가지고 있습니다. 운영체제는 먼저 루트 i-node를 읽어 루트 디렉터리의 데이터 블록 위치를 알아내고, 그 블록 안에서 다음 하위 디렉터리인 usr의 i-node 번호를 찾습니다. 이 과정을 파일이 있는 최종 위치까지 계층적으로 반복하여 파일의 i-node를 찾아내게 됩니다.

[복합문제 2] 파일 시스템 접근 경로 (디스크 블록 탐색 순서)

- 질문 예시: 경로명 /usr/dev/source/app.c 파일을 읽기 위해 디스크에서 액세스해야 하는 디렉터리 블록과 i-node의 전체적인 흐름은 어떻게 되는가?
- 정답: 디스크에서 데이터가 이동하고 참조되는 구체적인 순서는 다음과 같습니다.
 1. 루트 i-node $\rightarrow$ 2. 루트 디렉터리 블록
 2. /usr 명칭의 i-node $\rightarrow$ 4. /usr 디렉터리 블록
 3. /usr/dev 명칭의 i-node $\rightarrow$ 6. /usr/dev 디렉터리 블록
 4. /usr/dev/source 명칭의 i-node $\rightarrow$ 8. /usr/dev/source 디렉터리 블록
 5. /usr/dev/source/app.c 파일의 i-node $\rightarrow$ 10. /usr/dev/source/app.c 파일의 실제 데이터 블록

해설: 각 계층의 디렉터리 또한 하나의 특별한 '파일'입니다. 따라서 디렉터리 내부를 보기 위해서는 해당 디렉터리의 i-node를 먼저 확인한 뒤, i-node가 가리키는 **디렉터리 데이터 블록(파일 목록 명단이 적힌 블록)**을 읽어야만 다음 단계의 하위 파일 정보를 얻을 수 있습니다.

[복합문제 3] 파일을 열 때 디스크 i-node를 메모리에 적재(오픈)하는 이유

- 질문 예시: 왜 파일을 open()할 때 디스크에 있는 i-node를 커널 메모리 내의 '메모리 i-node 테이블'에 적재해 두는가?
- 정답: 성능 최적화와 디스크 I/O 병목 현상을 줄이기 위해서입니다. * i-node에는 파일 크기, 파일 블록의 위치 정보(인덱스) 등 파일의 핵심 메타 정보들이 들어있습니다.
 - 파일에 대해 읽기(read)나 쓰기(write) 연산이 진행되면 이 파일 메타 정보가 수시로, 반복적으로 액세스됩니다.
 - 만약 i-node를 메모리에 올리지 않고 디스크에 그대로 둔 채 작업을 하면, 실제 파일 데이터를 읽을 때뿐만 아니라 블록 위치를 조회하기 위해 디스크 내부의 i-node 영역을 매번 반복해서 읽어야 하므로 심각한 속도 저하가 발생합니다.
 - 따라서 파일을 열 때 디스크 i-node를 메모리(i-node 테이블)에 미리 적재해 두면, 이후 발생하는 모든 메타 정보 조회가 메모리 속도로 빠르게 처리되어 시스템 효율이 극대화됩니다.